

Take-Home Question — Frontend

Product Context & User Needs

ExampleHR has a module that serves as the primary interface for employees to request time off. The Human Capital Management (HCM) system (like Workday or SAP) remains the **"Source of Truth"** for employment data. ExampleHR's frontend must present balances and request workflows that feel instant and trustworthy to the user, while the underlying data is owned by a system ExampleHR does not control.

The Problem

Presenting balances that are *both fast and correct* is notoriously difficult when the source of truth lives elsewhere. If an employee sees "10 days available" and requests 2, the UI needs to give instant feedback — but the HCM may reject the request, or the displayed balance may already be stale because the HCM granted a work-anniversary bonus thirty seconds ago. The frontend has to resolve this tension gracefully without confusing the user or the manager approving the request.

User Personas

- **The Employee** — wants to see an accurate balance and get instant feedback when they submit a request. Should never be told "approved" and then later "actually, denied."
- **The Manager** — needs to approve requests with confidence that the balance shown is valid at the moment of approval, not minutes ago.

The Task

You are tasked with building the **Time-Off frontend** for ExampleHR. Your goal is to design the UI, state management, and data-fetching layer that present time-off balances and manage the lifecycle of a request end-to-end, while staying honest about the fact that HCM — not ExampleHR — owns the numbers.

Interesting challenges to handle

- ExampleHR is not the only thing that mutates HCM balances. On work anniversaries or at the start of the year, balances can refresh underneath a user who already has the app open. The UI must reconcile without surprising them.
- HCM exposes a **real-time API** for reading and writing a single balance (e.g. "1 day for `locationId=X, employeeId=Y`"). Treat it as the authoritative per-cell read.

- HCM also exposes a **batch endpoint** that returns the full corpus of balances across all dimensions. Useful for initial hydration and periodic reconciliation, but expensive.
- HCM *usually* returns a clear error when you try to file against an invalid dimension combination or insufficient balance — but not always. The frontend must be **defensive**: assume a success response can still be wrong, and design the UX so a late-arriving contradiction is recoverable.
- Balances are **per-employee, per-location**. A single employee may have several rows.

What to Build

A frontend application with:

- An **employee view** for seeing balances (per-location) and submitting time-off requests.
- A **manager view** for reviewing and approving/denying pending requests, with the balance context visible at decision time.
- A **data layer** that talks to mock HCM endpoints (see below), handles optimistic updates, reconciles with the source of truth, and degrades gracefully when HCM is slow, wrong, or silent.
- **Mock HCM endpoints** — build them as Next.js route handlers (or MSW handlers) with enough logic to simulate the interesting behaviors: the real-time read/write, the batch corpus endpoint, a work-anniversary bonus that fires on a timer or trigger, occasional silent failures, and occasional conflict responses.
- **Storybook stories** for every meaningful UI state: loading, empty, stale, optimistic-pending, optimistic-rolled-back, HCM-rejected, HCM-silently-wrong, balance-refreshed-mid-session, etc. Storybook is your proof that you've thought through the states, not just the happy path.

What Your Work Will Be Measured Against

- **Eng Spec** — a well-written Technical Requirement Document (TRD). List the challenges, propose a solution, and analyze alternatives considered. We especially want to see your reasoning about optimistic updates vs pessimistic, cache invalidation strategy, how you reconcile a background refresh with an in-flight user action, and how the component tree maps to these concerns.
- **Test Suite** — since you are using Agentic Development, the value of your work lies in the rigor of your tests. Storybook interaction tests, component tests, and integration tests against the mock HCM are all fair game. Make a deliberate choice about what kinds of tests guard what kinds of regressions, and defend it in the TRD. The goal is a system future contributors cannot silently break.
- **Deliverables**
 - Your TRD
 - Your code in a GitHub repository
 - Your test cases and proof of coverage

- A running Storybook (deployed, e.g. to Chromatic or Vercel, or runnable with a single command)

Guides

- **Go all-in on agentic development; do not write a single line of code yourself.** The value is in how precisely you specify the TRD and how thoroughly you design the test cases — the agent writes the code.
- Build the **mock HCM endpoints** as part of your test harness. They should have enough real logic to simulate balance changes, anniversary bonuses, the occasional silent failure, and the occasional insufficient-balance rejection. Consider deploying them (or at least making them trivially runnable locally) so your tests and Storybook can exercise the full matrix.
- Develop with **Next.js (App Router)** and **Storybook**. Pick your state-management and data-fetching tools deliberately and justify them in the TRD.
- Assume balances are **per-employee per-location**.